

HashSet with Open Addressing

Open Addressing, Linear Probing, Tombstones, Rehashing · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate-Advanced

Problem

Design a MyHashSet of integer keys using open addressing - all entries live in one array, and collisions are resolved by probing for the next slot rather than chaining. Implement linear probing fully; quadratic probing and double hashing are discussed as extensions.

Support add(key) (ignore duplicates), contains(key), and remove(key). Deletion uses tombstones (a deleted marker) so that probe sequences are not broken. Keep the load factor at or below 0.7, rehashing to roughly double the capacity when it is exceeded.

Learning objectives: weigh chaining against open addressing, master linear probing, handle deletion correctly with tombstones, and trigger rehashing as the table fills.

Operations & Rules

- ADD key - insert the key; if it already exists, do nothing.
- CONTAINS key - output true or false.
- REMOVE key - if present, replace the key with a tombstone (not an empty slot).
- Slots are marked empty (-1) or tombstone (-2); linear probing visits $(\text{hash} + i) \bmod \text{capacity}$ for $i = 0, 1, 2, \dots$
- Load factor: rehash to double the capacity whenever $\text{size} / \text{capacity}$ exceeds 0.7 after an insertion.

Input / Output Format

Input: Line 1: the initial capacity. Each later line: ADD key, CONTAINS key, or REMOVE key (read until end of input).

Output: One line per CONTAINS (true/false), then Size and Capacity of the final table.

Examples

Example 1

Input: cap=8; add(1); add(9); add(17); contains(9); remove(9); contains(9); add(9)

Output:

```
true
false
true
```

Explanation: 1, 9 and 17 all hash to bucket 1 and probe to 1, 2, 3; after removing 9 a tombstone marks its slot, then re-adding 9 reuses it.

Example 2

Input: cap=8; add(5); contains(5); remove(5); contains(5)

Output:

```
true
false
```

Explanation: 5 is present, then absent once removed.

Constraints

- $0 \leq \text{key} \leq 10^6$; up to 10,000 operations.
- Open addressing only - no built-in set types.
- Use tombstones for deletion; rehash keeps the load factor ≤ 0.7 .

Approach

Store keys directly in an array with two sentinels: empty (-1) and tombstone (-2). The hash is $\text{key} \bmod \text{capacity}$; linear probing scans $(\text{hash} + i) \bmod \text{capacity}$.

add probes for the key: if found, do nothing; remember the first tombstone seen, and stop at the first empty slot - insert into the remembered tombstone if any (reuse), otherwise the empty slot. contains stops at the first empty slot (key absent) and skips over tombstones.

remove finds the key and writes a tombstone, never an empty slot - an empty slot there would truncate the probe chain and hide keys inserted after it.

When $\text{size} / \text{capacity}$ exceeds 0.7, rehash: double the capacity, reset the table, and re-add only the real keys (tombstones and empties are dropped).

Extensions: quadratic probing visits $\text{hash} + i^2 \bmod \text{capacity}$ to break up primary clustering; double hashing visits $h_1(k) + i \cdot h_2(k)$ with a second hash such as $h_2(k) = 1 + (\text{key} \bmod (\text{capacity} - 1))$, spreading probes even further.

Solution

A complete, tested reference solution in each language. All three read the same input and produce identical output.

Python

```
# Lab 2 - MyHashSet with Open Addressing (linear probing + tombstones)
```

```

import sys

EMPTY = -1
TOMB = -2

class MyHashSet:
    def __init__(self, capacity=16):
        self.capacity = capacity
        self.size = 0
        self.table = [EMPTY] * capacity

    def _hash(self, key):
        return key % self.capacity

    def _probe(self, key, i):
        return (self._hash(key) + i) % self.capacity    # linear probing

    def add(self, key):
        first_tomb = -1
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.table[idx] == EMPTY:
                slot = first_tomb if first_tomb != -1 else idx
                self.table[slot] = key
                self.size += 1
                if self.size / self.capacity > 0.7:
                    self._rehash()
                return
            if self.table[idx] == TOMB:
                if first_tomb == -1:
                    first_tomb = idx
            elif self.table[idx] == key:
                return    # already present

    def contains(self, key):
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.table[idx] == EMPTY:
                return False
            if self.table[idx] == key:
                return True
        return False

    def remove(self, key):
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.table[idx] == EMPTY:
                return
            if self.table[idx] == key:
                self.table[idx] = TOMB
                self.size -= 1
                return

    def _rehash(self):
        old = self.table
        self.capacity *= 2

```

```

        self.table = [EMPTY] * self.capacity
        self.size = 0
        for v in old:
            if v != EMPTY and v != TOMB:
                self.add(v)

def main():
    lines = [ln for ln in sys.stdin.read().splitlines() if ln.strip()]
    cap = int(lines[0])
    s = MyHashSet(cap)
    out = []
    for line in lines[1:]:
        p = line.split()
        if p[0] == "ADD":
            s.add(int(p[1]))
        elif p[0] == "CONTAINS":
            out.append("true" if s.contains(int(p[1])) else "false")
        elif p[0] == "REMOVE":
            s.remove(int(p[1]))
    out.append("Size: " + str(s.size))
    out.append("Capacity: " + str(s.capacity))
    print("\n".join(out))

main()

```

C++

```

// Lab 2 - MyHashSet with Open Addressing (linear probing + tombstones)
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
using namespace std;
static const int EMPTY = -1, TOMB = -2;
class MyHashSet {
    int capacity, size;
    vector<int> table;
    int hashKey(int key){ return key % capacity; }
    int probe(int key, int i){ return (hashKey(key) + i) % capacity; }
    void rehash(){
        vector<int> old = table;
        capacity *= 2;
        table.assign(capacity, EMPTY);
        size = 0;
        for (int v : old) if (v != EMPTY && v != TOMB) add(v);
    }
public:
    MyHashSet(int cap): capacity(cap), size(0), table(cap, EMPTY) {}
    void add(int key){
        int firstTomb = -1;
        for (int i = 0; i < capacity; i++){
            int idx = probe(key, i);
            if (table[idx] == EMPTY){
                int slot = (firstTomb != -1) ? firstTomb : idx;
                table[slot] = key; size++;
                if ((double)size / capacity > 0.7) rehash();
            }
        }
        return;
    }
};

```

```

        }
        if (table[idx] == TOMB){ if (firstTomb == -1) firstTomb = idx; }
        else if (table[idx] == key) return;
    }
}
bool contains(int key){
    for (int i = 0; i < capacity; i++){
        int idx = probe(key, i);
        if (table[idx] == EMPTY) return false;
        if (table[idx] == key) return true;
    }
    return false;
}
void remove(int key){
    for (int i = 0; i < capacity; i++){
        int idx = probe(key, i);
        if (table[idx] == EMPTY) return;
        if (table[idx] == key){ table[idx] = TOMB; size--; return; }
    }
}
int getSize(){ return size; }
int getCapacity(){ return capacity; }
};
int main(){
    string line; getline(cin, line);
    int cap = stoi(line);
    MyHashSet s(cap);
    vector<string> out;
    while (getline(cin, line)){
        if (line.empty()) continue;
        istringstream iss(line); string op; int k; iss >> op >> k;
        if (op == "ADD") s.add(k);
        else if (op == "CONTAINS") out.push_back(s.contains(k) ? "true" : "false");
        else if (op == "REMOVE") s.remove(k);
    }
    out.push_back("Size: " + to_string(s.getSize()));
    out.push_back("Capacity: " + to_string(s.getCapacity()));
    for (auto& x : out) cout << x << "\n";
    return 0;
}

```

Java

```

// Lab 2 - MyHashSet with Open Addressing (linear probing + tombstones)
import java.util.*;
public class Main {
    static final int EMPTY = -1, TOMB = -2;
    static class MyHashSet {
        int capacity, size = 0;
        int[] table;
        MyHashSet(int cap){ capacity = cap; table = new int[cap]; Arrays.fill(table,
EMPTY); }
        int hashKey(int key){ return key % capacity; }
        int probe(int key, int i){ return (hashKey(key) + i) % capacity; }
        void add(int key){
            int firstTomb = -1;
            for (int i = 0; i < capacity; i++){

```

```

        int idx = probe(key, i);
        if (table[idx] == EMPTY){
            int slot = (firstTomb != -1) ? firstTomb : idx;
            table[slot] = key; size++;
            if ((double) size / capacity > 0.7) rehash();
            return;
        }
        if (table[idx] == TOMB){ if (firstTomb == -1) firstTomb = idx; }
        else if (table[idx] == key) return;
    }
}

boolean contains(int key){
    for (int i = 0; i < capacity; i++){
        int idx = probe(key, i);
        if (table[idx] == EMPTY) return false;
        if (table[idx] == key) return true;
    }
    return false;
}

void remove(int key){
    for (int i = 0; i < capacity; i++){
        int idx = probe(key, i);
        if (table[idx] == EMPTY) return;
        if (table[idx] == key){ table[idx] = TOMB; size--; return; }
    }
}

void rehash(){
    int[] old = table;
    capacity *= 2;
    table = new int[capacity]; Arrays.fill(table, EMPTY);
    size = 0;
    for (int v : old) if (v != EMPTY && v != TOMB) add(v);
}

}

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    int cap = Integer.parseInt(sc.nextLine().trim());
    MyHashSet s = new MyHashSet(cap);
    StringBuilder sb = new StringBuilder();
    while (sc.hasNextLine()){
        String line = sc.nextLine().trim();
        if (line.isEmpty()) continue;
        String[] p = line.split("\\s+");
        int k = Integer.parseInt(p[1]);
        if (p[0].equals("ADD")) s.add(k);
        else if (p[0].equals("CONTAINS")) sb.append(s.contains(k) ? "true" :
"false").append("\n");
        else if (p[0].equals("REMOVE")) s.remove(k);
    }
    sb.append("Size: ").append(s.size).append("\n");
    sb.append("Capacity: ").append(s.capacity).append("\n");
    System.out.print(sb);
}
}

```

Sample Run

Sample Input

```
8
ADD 1
ADD 9
ADD 17
CONTAINS 9
REMOVE 9
CONTAINS 9
ADD 9
ADD 2
ADD 10
ADD 3
CONTAINS 17
CONTAINS 99
```

Sample Output

```
true
false
true
false
Size: 6
Capacity: 16
```

Follow-Up & Discussion

- Open addressing has better cache locality (one contiguous array) and no per-node pointers, but suffers clustering and needs tombstones; chaining has simpler deletion and no clustering, but poorer locality and pointer overhead.
- Linear probing causes primary clustering: a run of occupied slots makes any key hashing nearby join and extend the same run, so clusters grow and lengthen probe sequences.
- Double hashing beats quadratic in a Fintech high-frequency lookup table when you need probe sequences that differ per key (quadratic keys sharing a home slot still follow the same offsets - secondary clustering); double hashing's key-dependent step largely removes that.
- Load factor bites harder under open addressing: as it nears 1 the expected probe count explodes (roughly $1/(1 - \text{load})$), whereas chaining degrades linearly with the average chain length, which is why open addressing rehashes at a lower threshold (0.7 here).